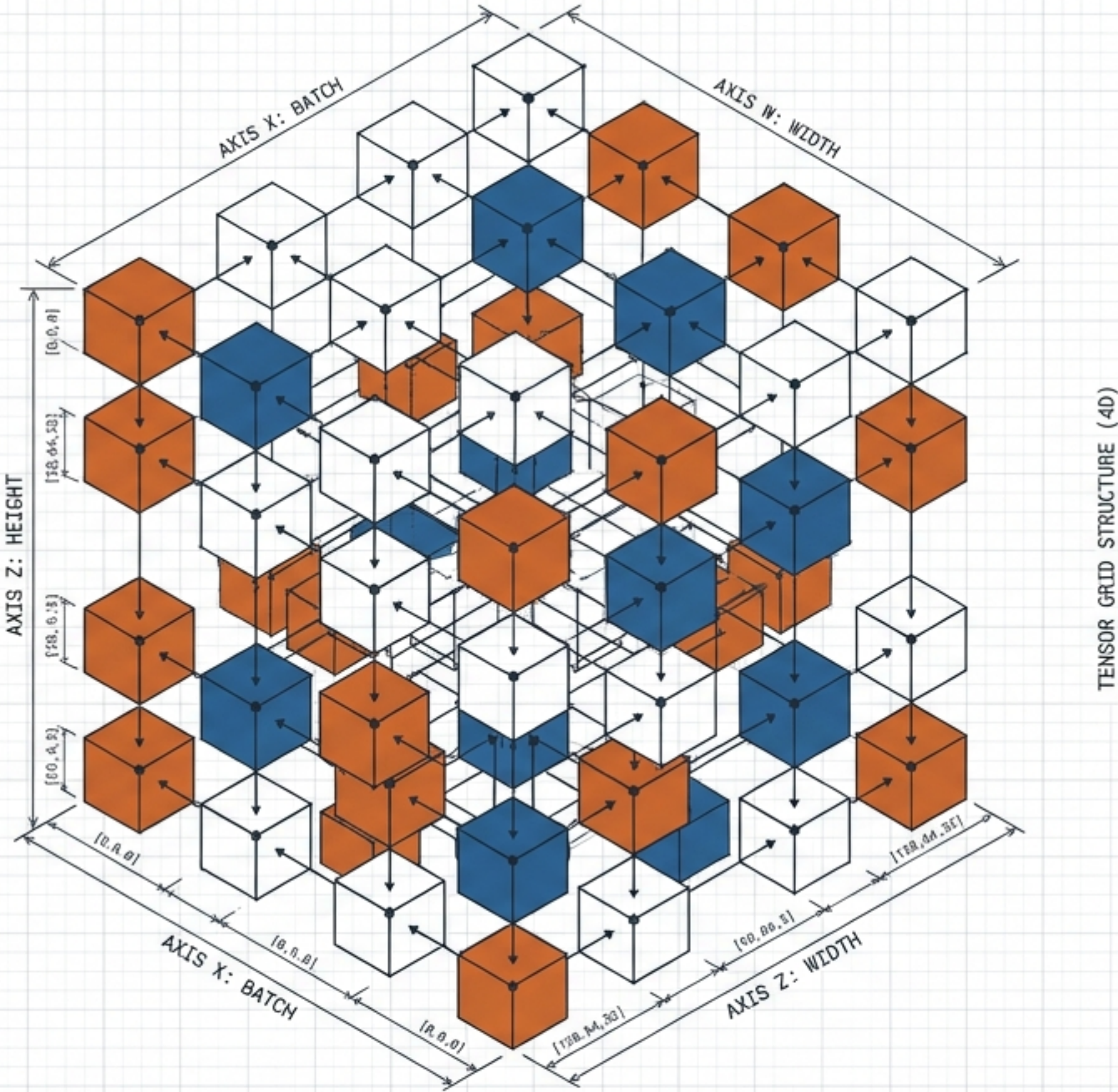


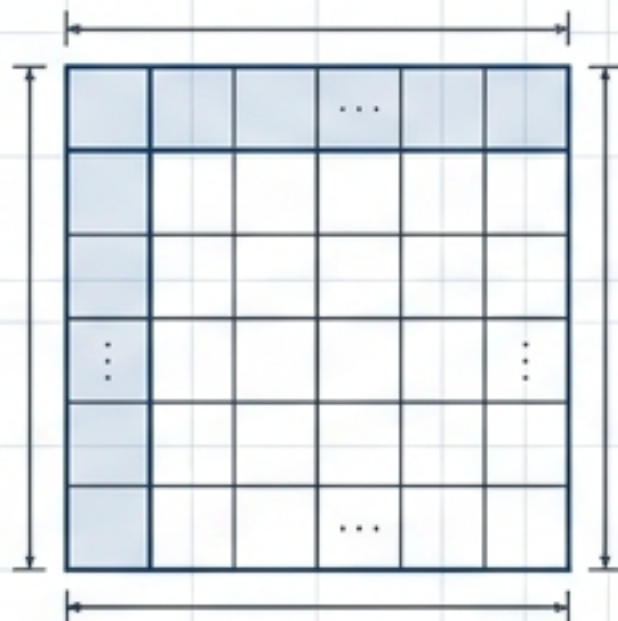
GPU Tensor Core Performance Atlas

A Workload Diagnostic:
CUDA Rust vs. Legacy CUDA
Fortran

// DATASET: V5-2 | METRIC: GFLOPS | ✓ PROFILER_READY

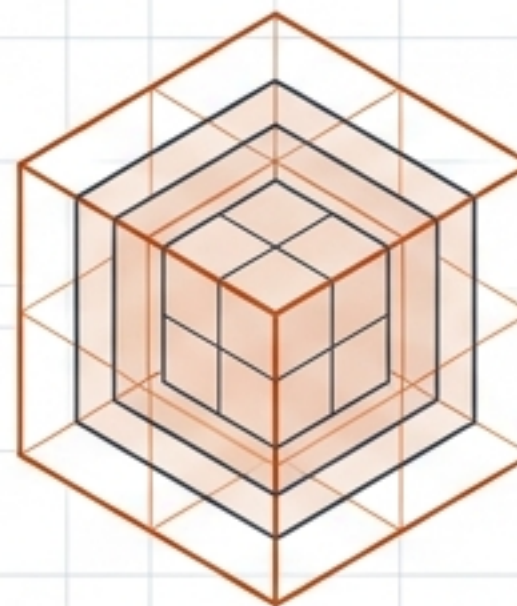


Raw BLAS Math



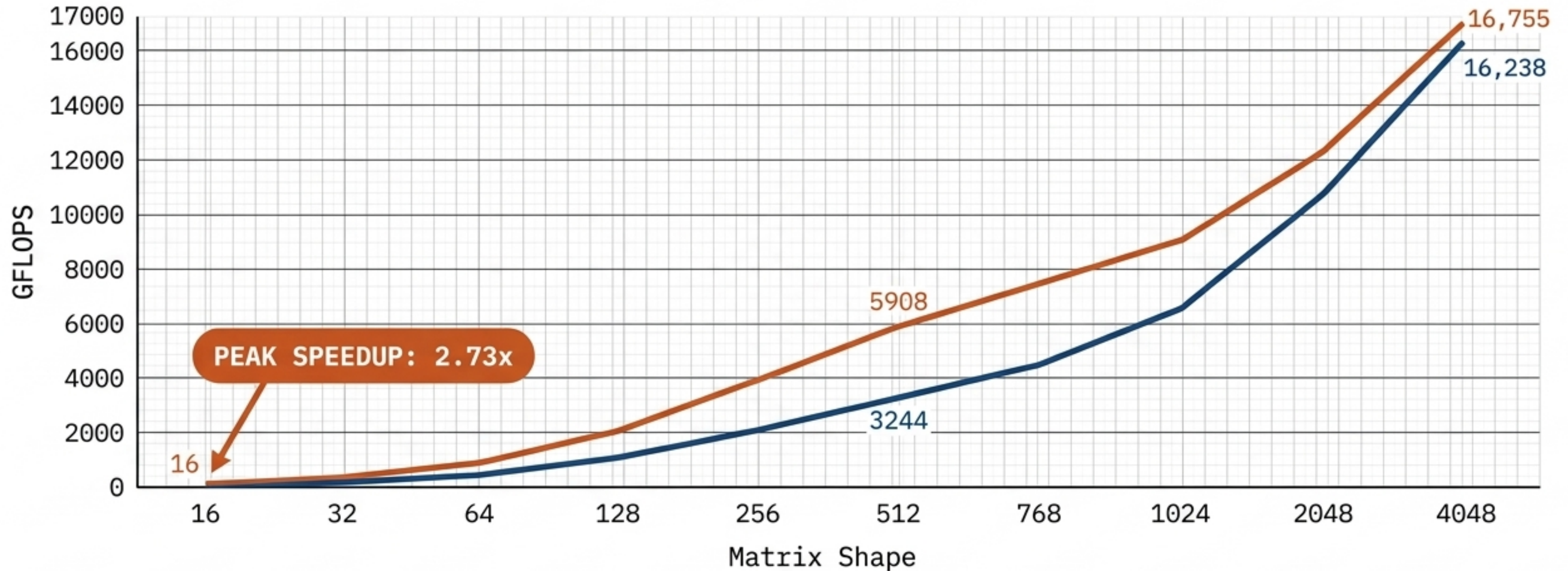
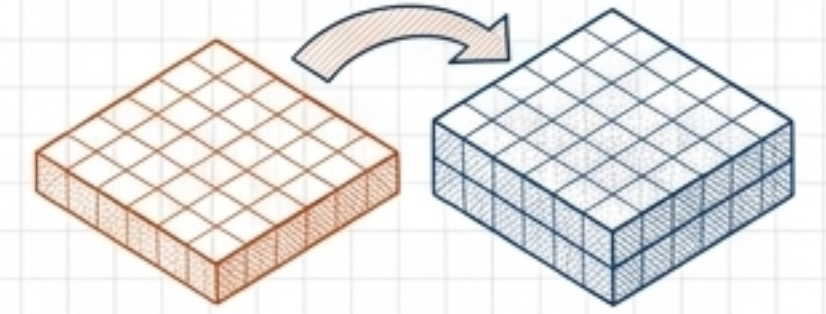
- Legacy Fortran bindings dominate pure, unbatched linear algebra.
- Heavily outpaces Rust in flat `matrix_multiply` (up to 2x faster at 512^3).
- Maintains consistent 10-20% edges in `vector_matrix` operations.

Complex & Batched Control Flow



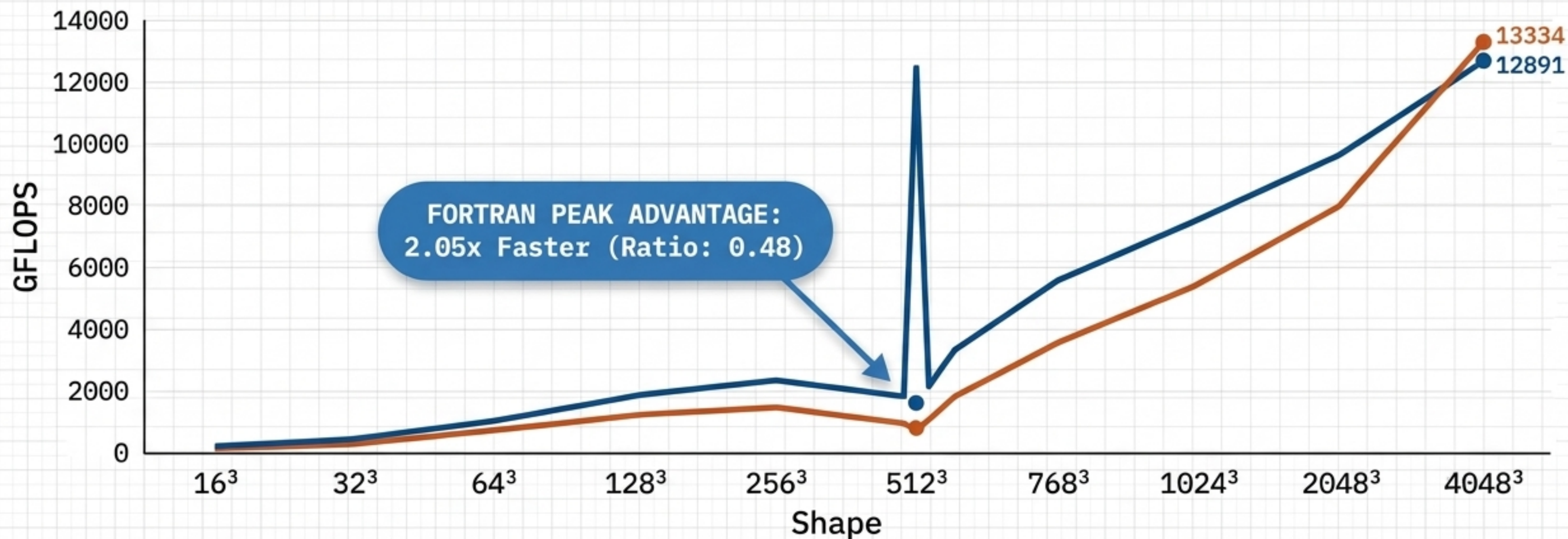
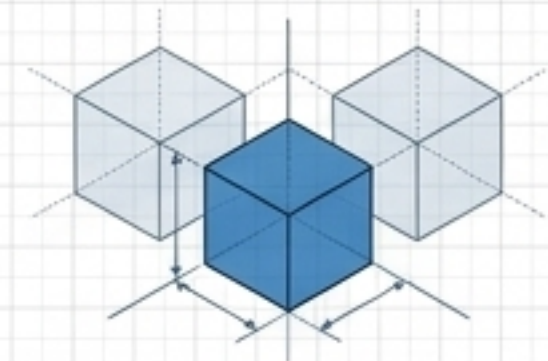
- Rust excels at GPU loop management and batched state operations.
- Delivers **2.73x** speedup in `matrix_power`.
- Achieves **2.32x** speedup in `batched_vector`.
- Maintains near 2x scaling advantage in `strided_batch`.

Operation Profiler: matrix_power



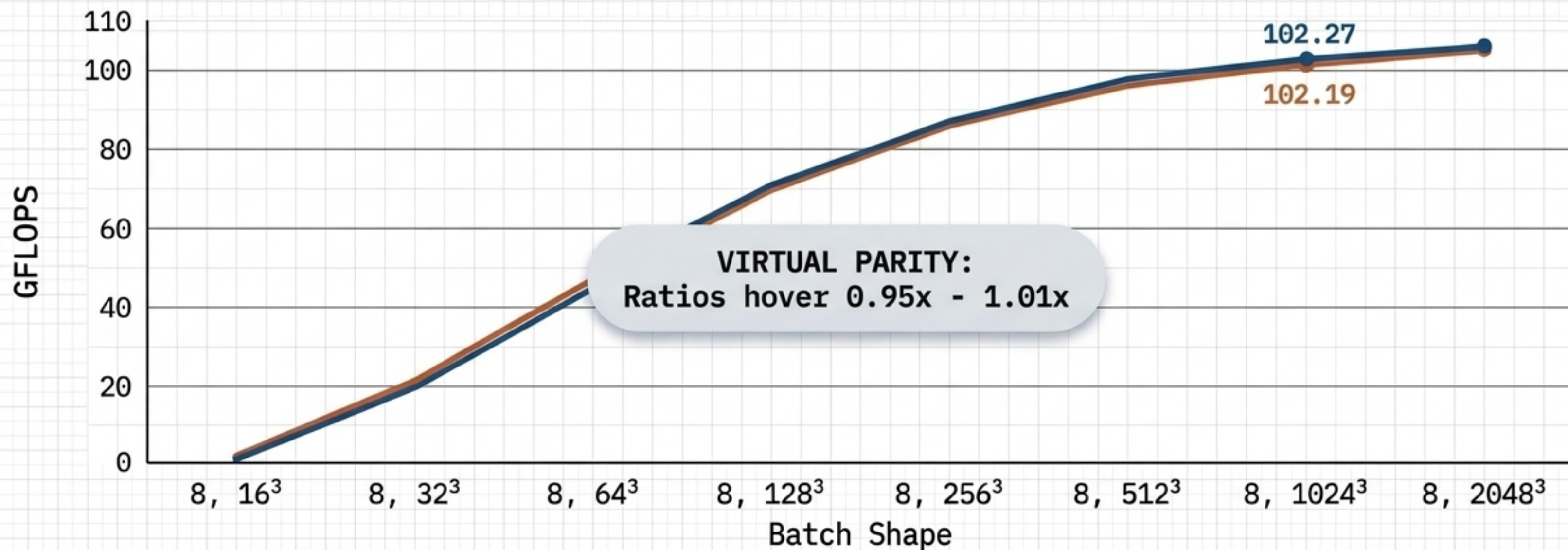
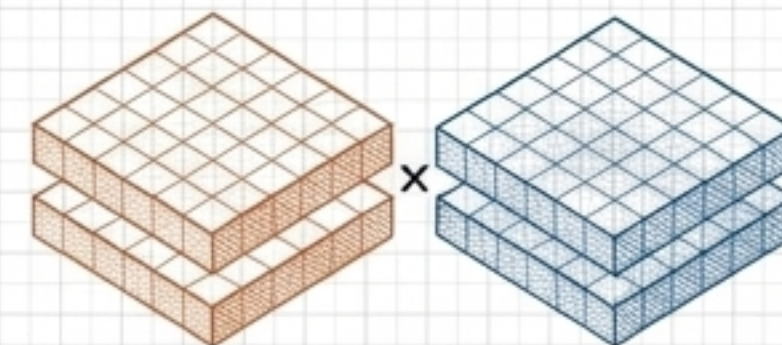
Rust dominates mid-scale iterative matrix power loops, generating massive throughput advantages before converging with Fortran at extreme memory-bound limits (4048^2).

Operation Profiler: matrix_multiply



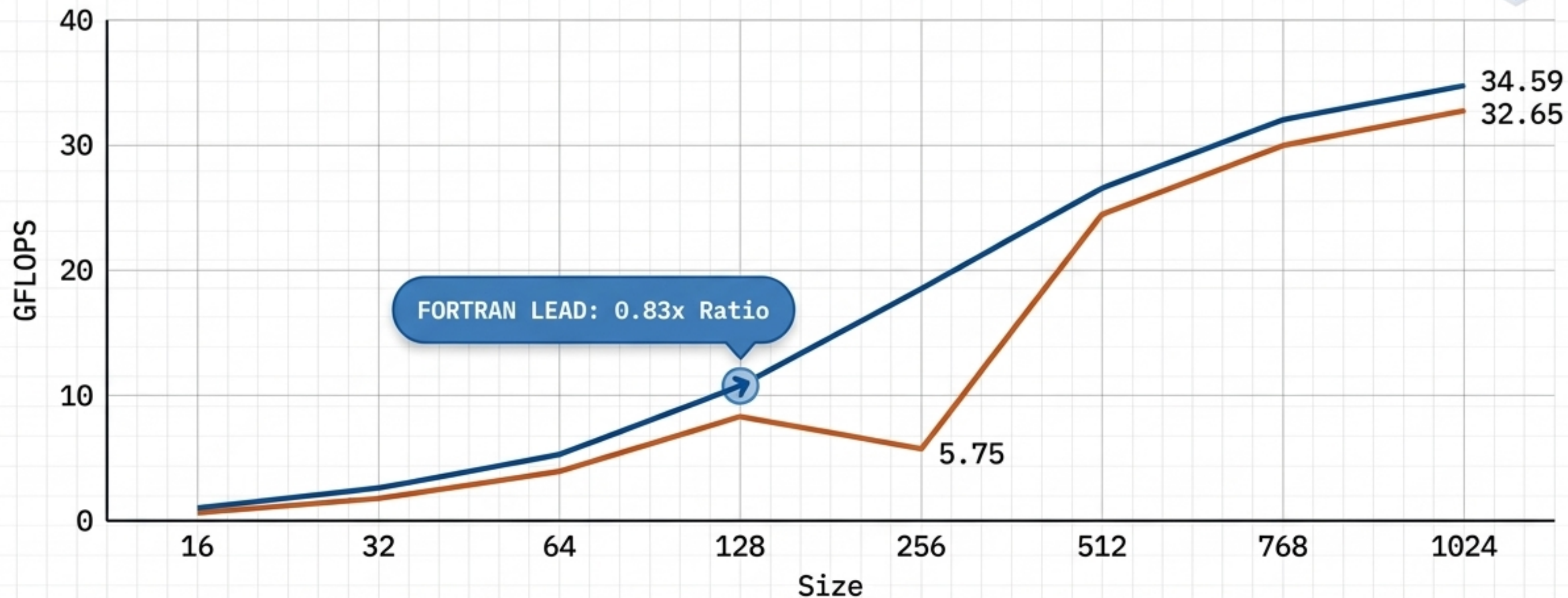
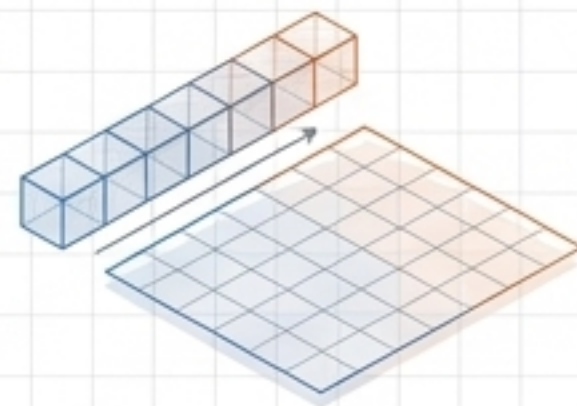
For standard 3D matrix multiplication, Fortran's mature BLAS engine is virtually untouchable in the mid-scale, processing over double the GFLOPS of Rust at 512^3 .

Operation Profiler: `batched_matmul_fp64`



High-precision (fp64) batched matrix multiplication is strictly compute-bound by the Tensor Cores. Both languages yield identical maximum throughput (~104 GFLOPS), rendering the host language irrelevant to performance.

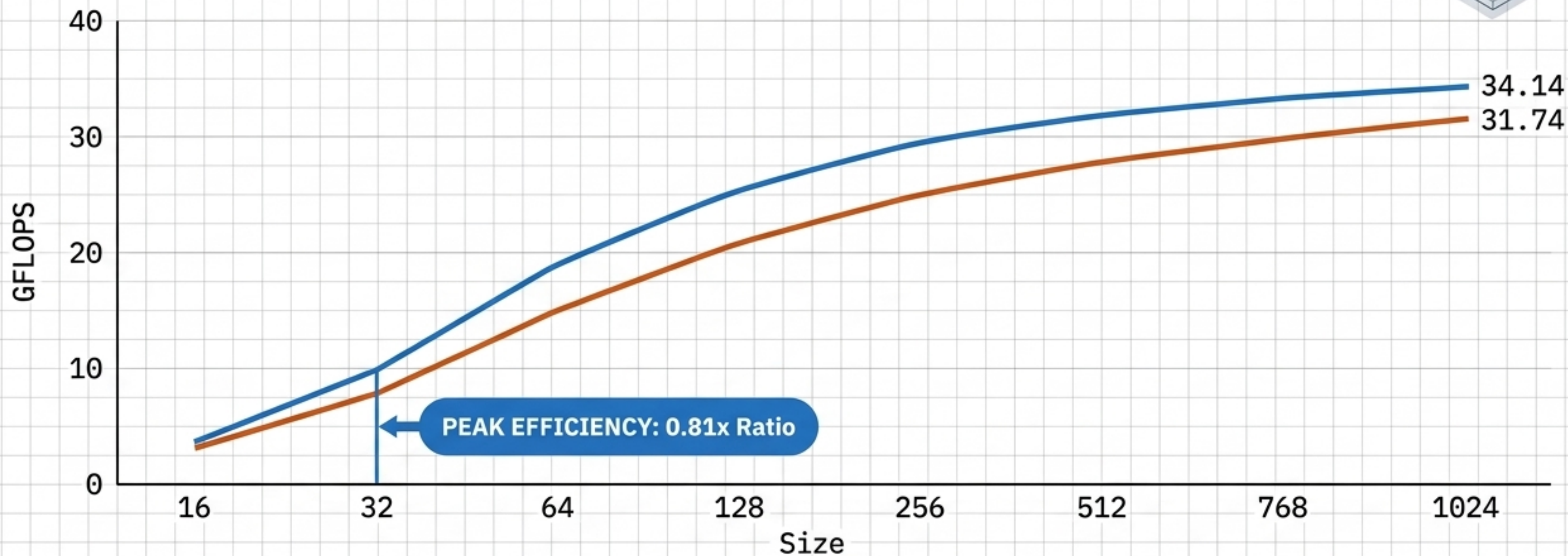
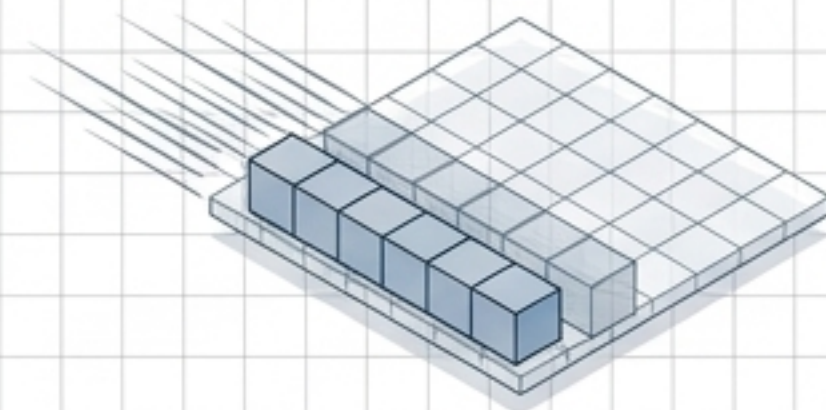
Operation Profiler: vector_matrix



Fortran maintains a reliable 5-15% throughput advantage in standard vector-matrix operations, processing slightly more efficiently across almost all dimensional scales (with an anomalous Rust spike at 256).

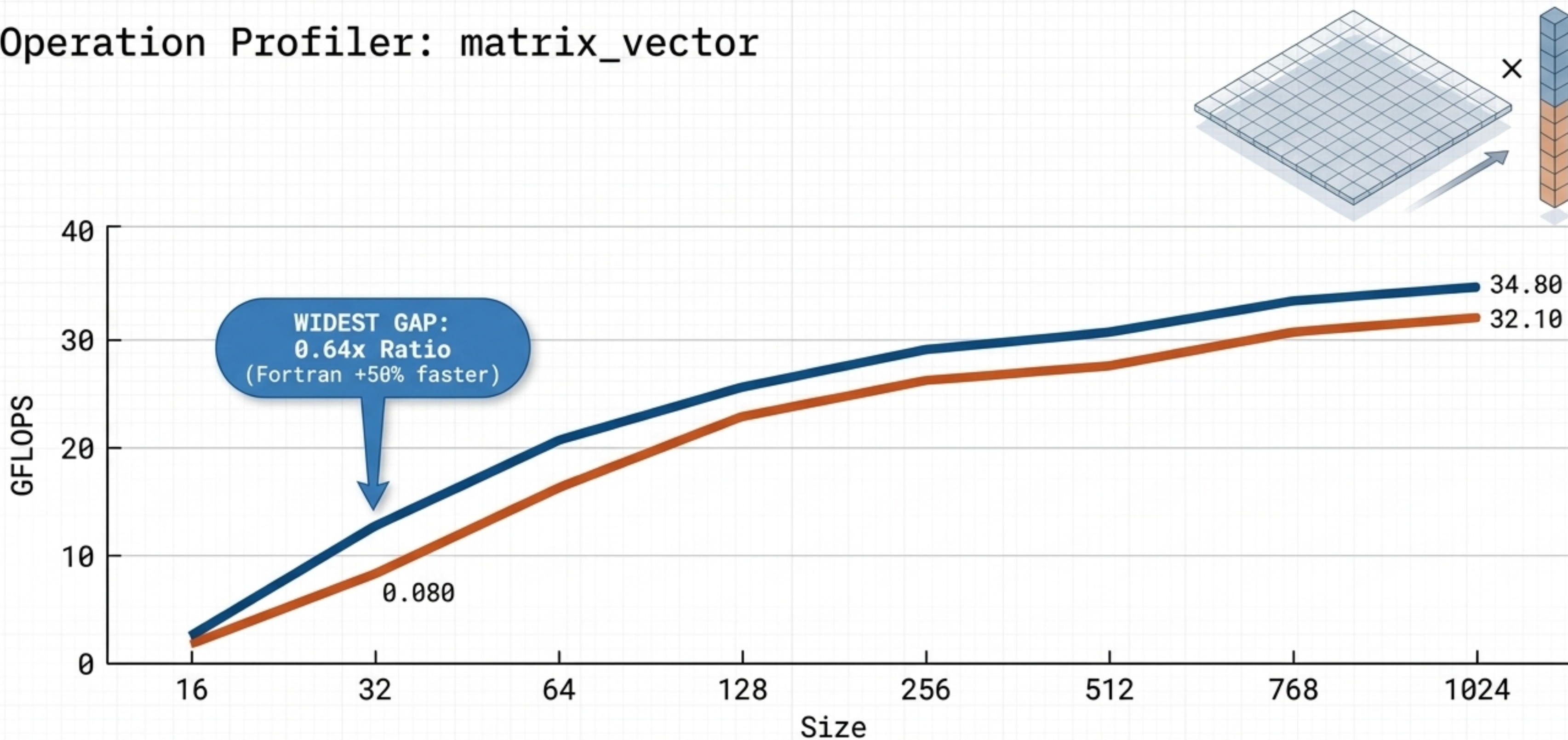
[RUST (ORANGE) vs FORTRAN (BLUE) - Y-AXIS: GFLOPS]

Operation Profiler: vector_matrix_optimised



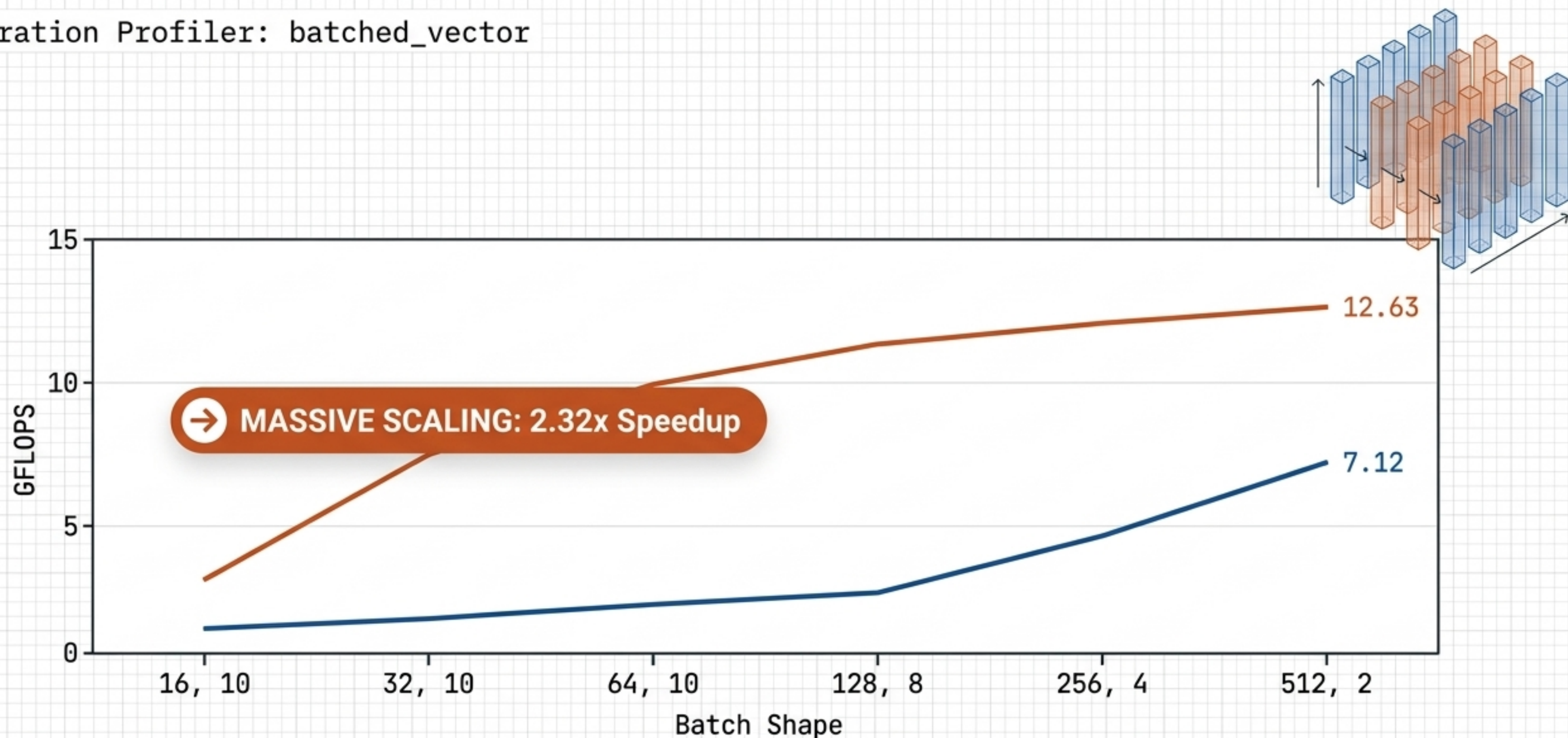
Applying optimized execution paths solidifies legacy dominance. Fortran outperforms Rust at every single recorded scale, maximizing its architectural advantage in lower-dimensional linear algebra.

Operation Profiler: matrix_vector



The reversal of operands changes nothing; flat matrix-vector math belongs to Fortran. The legacy bindings extract up to 50% more throughput at small scales and maintain the lead through size 1024.

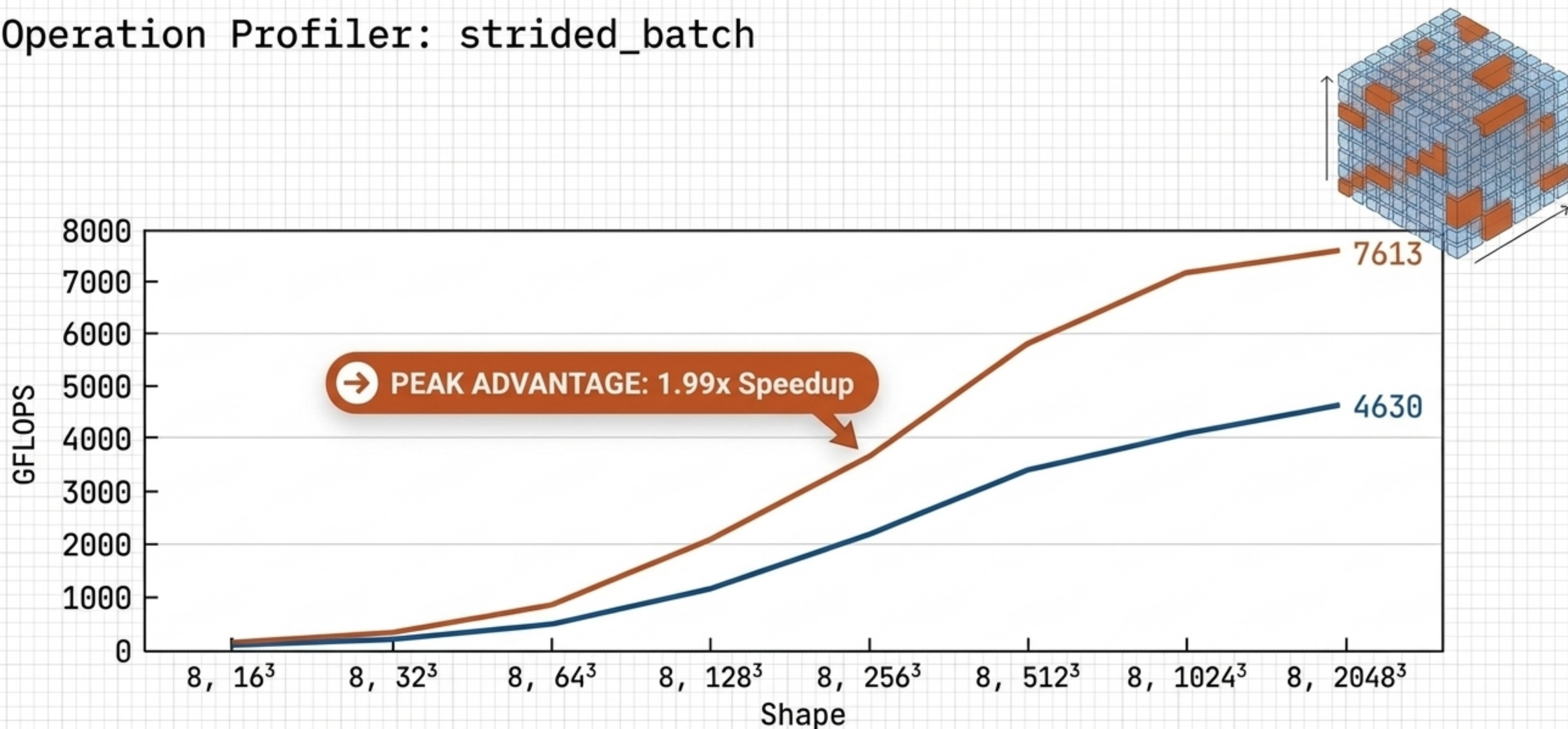
Operation Profiler: batched_vector



When single vectors are processed in rapid batches, Rust's low-overread dispatch engine takes over. Rust consistently pushes nearly **double the GFLOPS** of Fortran across all tested batch shapes.

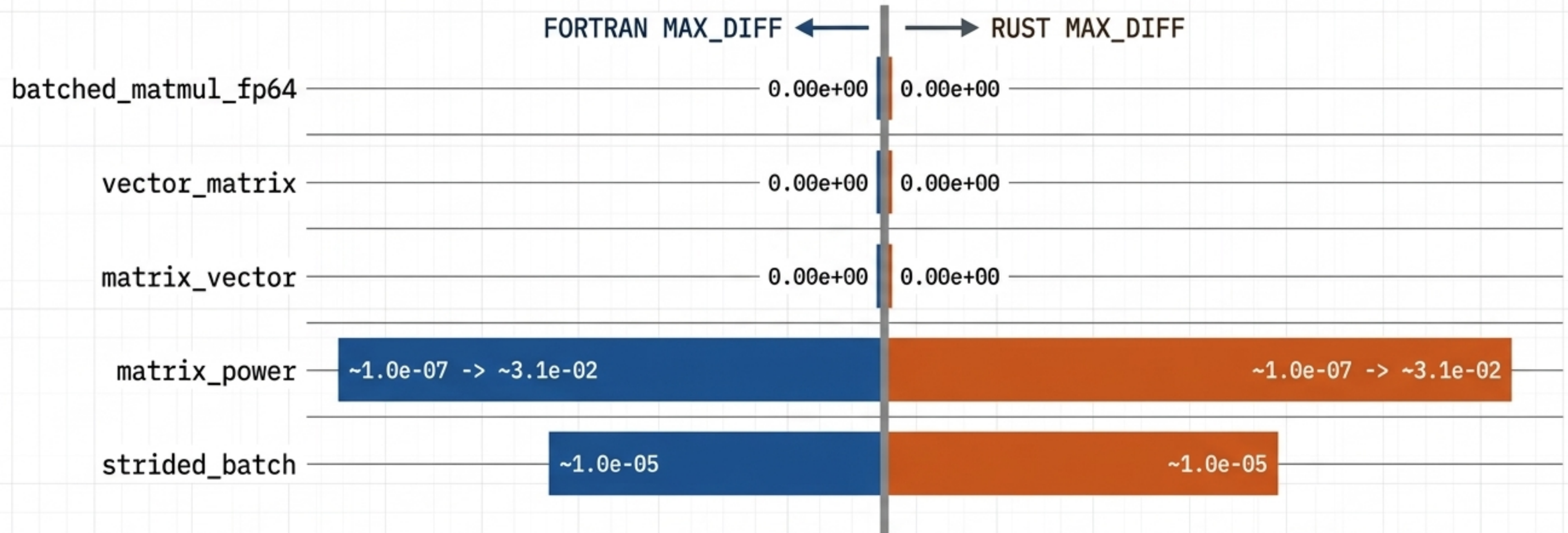
[RUST (ORANGE) vs FORTRAN (BLUE) - Y-AXIS: GFLOPS]

Operation Profiler: strided_batch



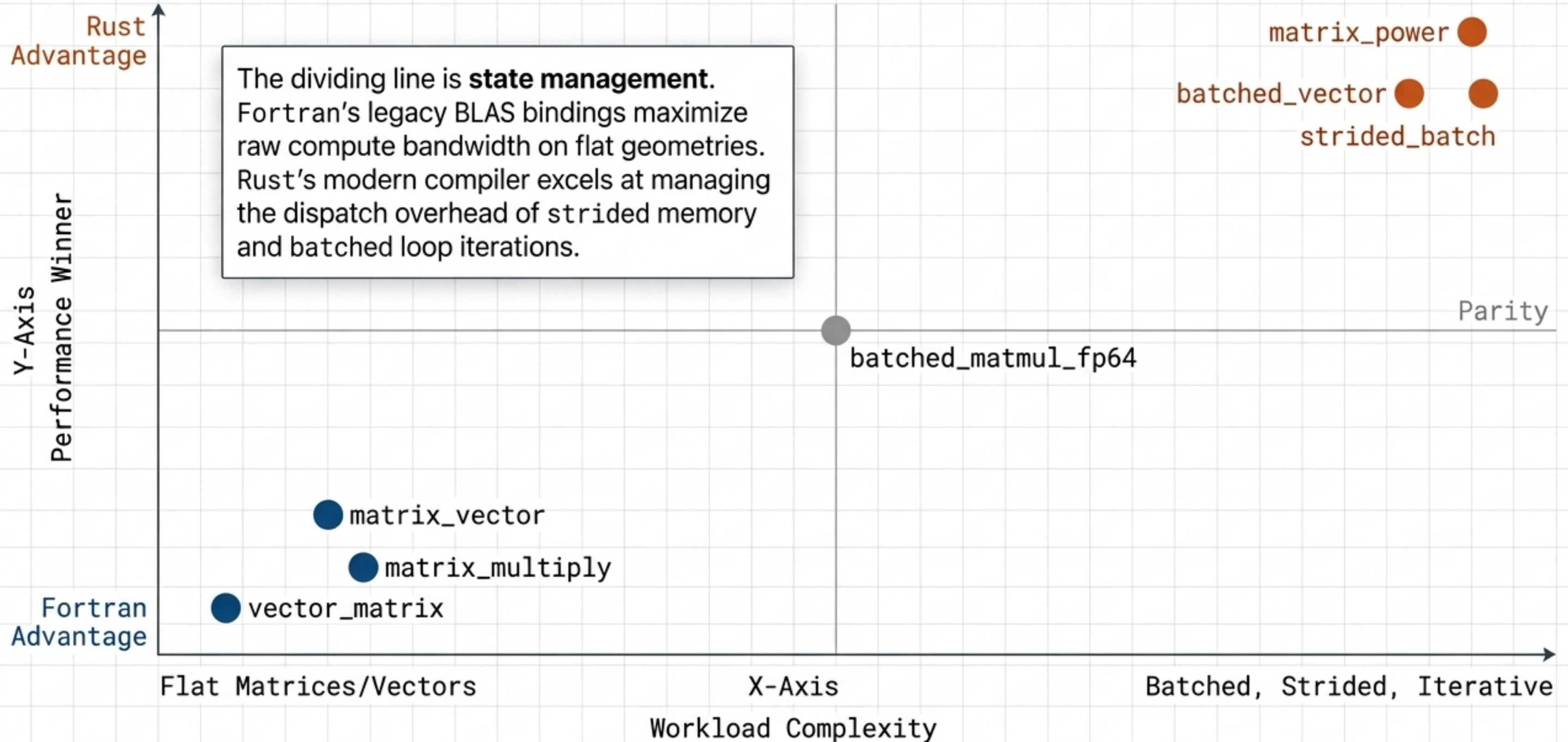
Memory layout matters. When batching requires jumping across memory strides, Rust's superior pointer and loop semantics effectively **double the throughput** of Fortran at optimal mid-scales (256³).

Precision Telemetry: Floating Point Variance



Performance gains require zero precision trade-offs. The max_diff metrics between Fortran and Rust are statistically mirrored across all 65 test configurations. Where speedups occur, the math remains exact.

The Architectural Landscape



Language Selection Matrix

Operation Class	Recommended Target	Peak Ratio	Scaling Behavior
matrix_power	RUST	2.73x	Rust dominates mid-scale, converges at high scale.
batched_vector	RUST	2.32x	Rust universally pushes 2x throughput.
strided_batch	RUST	1.99x	Rust advantage expands alongside tensor size.
batched_matmul_fp64	EITHER	1.01x	Hardware bottlenecked; pure parity.
matrix_multiply	FORTRAN	0.48x	Legacy bindings double throughput at 512 ³ .
vector_matrix	FORTRAN	0.83x	Fortran maintains consistent 10-15% edge.
matrix_vector	FORTRAN	0.64x	Fortran yields up to 50% faster flat dispatches.

// END PROFILER REPORT. BASELINE ARCHITECTURE DECISIONS ON WORKLOAD COMPLEXITY.